

# RSM - Agente Funcional de Gestión de Inventario

Informe técnico EP2 | ISY0101 | Versión funcional en Visual Studio Code

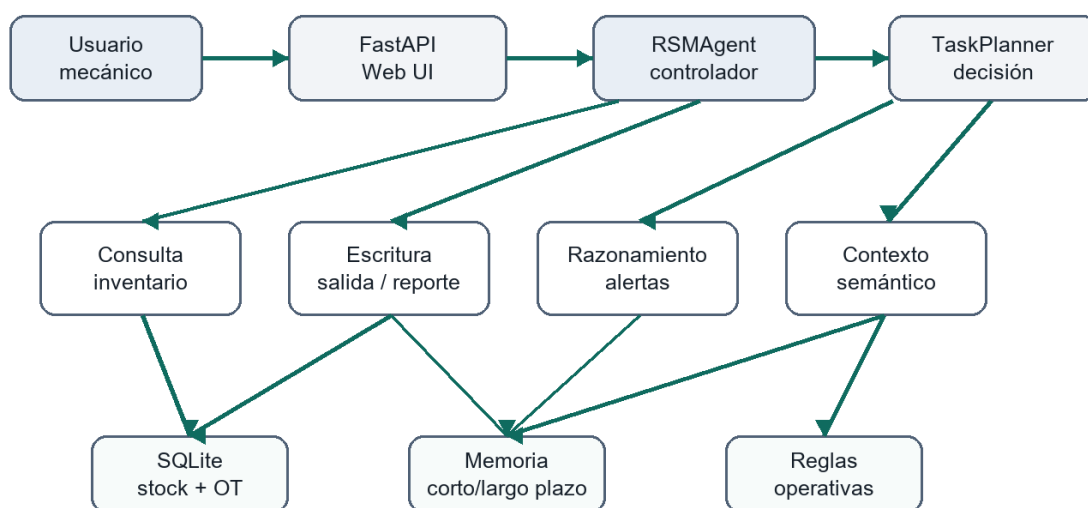
| Dato              | Detalle                                                                          |
|-------------------|----------------------------------------------------------------------------------|
| Proyecto          | Sistema inteligente RSM para consultas, trazabilidad y alertas de stock.         |
| Repositorio local | Carpeta rsm-agente-ep2, lista para abrirse en VS Code o subirse a GitHub/GitLab. |
| Stack             | Python 3, FastAPI, SQLite, LangChain Core, HTML/CSS/JS y pytest.                 |

## 1. Propósito y alcance

El proyecto transforma el diseño de EP1 en un MVP funcional para el taller RSM. El agente permite consultar inventario en lenguaje natural, registrar salidas con orden de trabajo, detectar quiebres de stock y redactar evidencia operativa. La solución opera localmente para respetar la restricción de privacidad del caso y evitar dependencia de APIs pagadas.

Flujo principal: el mecánico envía una solicitud, FastAPI la entrega al agente, el planificador decide el flujo y las herramientas LangChain consultan o escriben datos en SQLite. La respuesta incluye plan, herramientas utilizadas y memoria reciente, por lo que el evaluador puede validar la orquestación.

### Orquestación del agente RSM EP2



Evidencia: el agente devuelve plan, herramientas usadas, memoria reciente y respuesta con fuente.

## 2. Diseño e implementación del agente

La implementación se separa en cinco módulos: API, agente, planificador, herramientas y persistencia. Esta separación permite probar cada responsabilidad y demostrar autonomía sin ocultar la lógica en un script único.

| Componente       | Rol técnico                                                                               | Evidencia                       |
|------------------|-------------------------------------------------------------------------------------------|---------------------------------|
| app/main.py      | Expone API REST, interfaz web y endpoints de salud, inventario, chat, memoria y reportes. | FastAPI + /docs                 |
| app/agent.py     | Orquesta plan, herramientas y memoria para cada mensaje del usuario.                      | Respuesta con plan y tools_used |
| app/tools.py     | Define herramientas LangChain de consulta, escritura, contexto y alertas.                 | StructuredTool                  |
| app/database.py  | Gestiona SQLite, inventario, movimientos y memoria persistente.                           | data/rsm_agent.db               |
| app/retriever.py | Recupera contexto local desde inventario, reglas y memoria por similitud.                 | hits con score                  |

### Herramientas integradas

- Consulta: consultar\_inventario recupera repuestos, stock, ubicación, proveedor y compatibilidad.
- Escritura: registrar\_salida descuenta stock con trazabilidad y redactar\_reporte\_alertas genera evidencia en outputs.
- Razonamiento: generar\_alertas clasifica CRÍTICO o BAJO y recomienda reposición según prioridad.

### 3. Memoria, contexto y planificación

La memoria corta se representa con los últimos eventos de la sesión; la memoria larga queda persistida en la tabla memory de SQLite. Cada salida registrada guarda una huella con OT, mecánico, vehículo, repuesto y stock resultante. La recuperación de contexto combina tres fuentes: inventario vigente, reglas operativas y memoria histórica de la sesión.

| Escenario          | Decisión adaptativa del agente                    | Resultado esperado                                  |
|--------------------|---------------------------------------------------|-----------------------------------------------------|
| Consulta de stock  | Busca contexto relevante y responde con fuente.   | Stock, ubicación, proveedor y compatibilidad.       |
| Salida incompleta  | Detecta campos faltantes antes de escribir.       | No descuenta stock y solicita OT/mecánico/vehículo. |
| Stock insuficiente | Valida disponibilidad antes del movimiento.       | Rechaza la salida y conserva inventario.            |
| Stock bajo mínimo  | Registra salida y agrega advertencia operacional. | Recomienda reposición al proveedor.                 |

#### Planificación de tareas

El módulo app/planner.py clasifica la intención en consultar\_stock, registrar\_salida, generar\_alertas, recomendar\_reposicion o redactar\_reporte. Para cada intención secuencia pasos: validar datos, consultar contexto, ejecutar herramienta, registrar memoria y responder con evidencia.

## 4. Evidencia de funcionamiento

El proyecto fue preparado para Visual Studio Code con archivo .code-workspace, launch.json y tasks.json. El evaluador puede instalar dependencias, iniciar la API y ejecutar pruebas desde Terminal > Run Task o con F5.

| Prueba                  | Comando o entrada                           | Evidencia                                            |
|-------------------------|---------------------------------------------|------------------------------------------------------|
| Tests automatizados     | .\.venv\Scripts\python.exe -m pytest -q     | 6 passed                                             |
| Consulta                | ¿Cuántos filtros de aceite Toyota quedan?   | Usa recuperar_contexto y consultar_inventario.       |
| Salida valida           | Salida de 2 unidades FO-TOY-001 para OT-778 | Stock baja de 8 a 6 y se guarda memoria.             |
| Salida sin trazabilidad | Registra salida de 2 unidades FO-TOY-001    | Bloquea escritura por falta de OT/mecánico/vehículo. |
| Alertas                 | ¿Qué ítems están críticos?                  | Entrega ítems CRÍTICO/BAJO y proveedor sugerido.     |

### Instrucciones de ejecución

- Abrir rsm-agente-ep2.code-workspace en Visual Studio Code.
- Ejecutar las tareas: Instalar dependencias, Inicializar base de datos y Ejecutar API.
- Probar la interfaz en <http://127.0.0.1:8000> y la documentación en /docs.

## 5. Cierre y referencias

La versión EP2 cumple el paso clave que faltaba en el diseño inicial: existe un agente ejecutable, verificable y con persistencia real. El sistema no simula solo una respuesta de IA; aplica reglas, consulta datos, escribe movimientos y conserva memoria. La solución puede crecer incorporando un LLM local por Ollama, ChromaDB o un frontend React, pero el MVP actual ya permite validar autonomía funcional y continuidad de tarea.

| Indicador | Cumplimiento                                                          |
|-----------|-----------------------------------------------------------------------|
| IE1-IE2   | Herramientas autónomas implementadas con LangChain Core.              |
| IE3-IE4   | Memoria persistente y recuperación de contexto local.                 |
| IE5-IE6   | Planificación por intención y decisiones ante condiciones cambiantes. |
| IE7-IE10  | README, diagrama, pruebas, informe técnico y referencias.             |

### Referencias

FastAPI. (2024). FastAPI documentation. <https://fastapi.tiangolo.com/>

LangChain. (2024). LangChain documentation. <https://python.langchain.com/docs/>

Python Software Foundation. (2024). Python documentation. <https://docs.python.org/>

SQLite. (2024). SQLite documentation. <https://sqlite.org/docs.html>